

Partie I : Graphopolis

Bienvenue à Graphopolis

Graphopolis est une ville imaginaire où les rues relient des carrefours. Les habitants veulent organiser des fêtes de quartier, mais sans embouteillages : deux fêtes ne peuvent pas avoir lieu à des carrefours reliés par une rue. Choisir les carrefours compatibles devient un vrai casse-tête : un **problème algorithmique**.

Définitions préliminaires

Un graphe non orienté est défini par $G = (V, E)$, où :

- V est l'ensemble des sommets (les carrefours) ;
- E est l'ensemble des arêtes (les rues).

Un **ensemble stable** (ou *independent set*) est un sous-ensemble $S \subseteq V$ tel que, pour tout couple $(u, v) \in S \times S$, on ait $(u, v) \notin E$. En d'autres termes, aucun lien direct ne relie deux sommets de S .

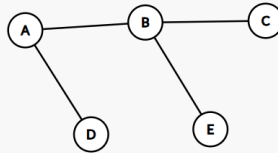
Graphopolis

Considérons le graphe de Graphopolis suivant :

Les rues sont : $(A, B), (B, C), (A, D), (B, E)$.

Des ensembles stables possibles incluent $\{A, C, E\}, \{D, C\}, \{D, E\}$.

Le plus grand stable contient 3 sommets.



Exercice 1 : Une Machine de Turing à Graphopolis

Question 1 :

Dans un premier temps, expliquer textuellement, s'il peut exister une machine de Turing qui décide des ensembles stables, étant donné un graphe $G = (V, E)$.

Question 2 :

Décrire et expliciter une machine de Turing M qui décide le problème de l'ensemble stable. Quels états et transitions utilise-t-elle ? Estimer son nombre d'actions élémentaires en fonction de $n = |V|$ et $m = |E|$.

Exercice 2 : Algorithmique

Pour évaluer l'efficacité d'un algorithme, on compte le nombre d'actions atomiques exécutées en fonction de la taille du graphe.

Considérons cet algorithme

```
pour chaque sous-ensemble S de V :  
  stable := vrai  
  pour chaque arête (u,v) de E :  
    si u et v sont tous deux dans S :  
      stable := faux  
  si stable et |S| >= k :  
    retourner vrai  
retourner faux
```

On considérera que le test de savoir si u est dans E est l'opération élémentaires de cet algorithme.

Question 1 :

Exprimer le coût total $f(n, m)$ de cet algorithme en nombre d'actions élémentaires, avec n le nombre de sommet, et m le nombre d'arrete (ou de rue). Donner sa complexité asymptotique (notation O).

Question 2 :

Discuter du caractère "raisonnable" de cet algorithme pour $n = 10, 20, 40$. Que devient le temps de calcul si chaque action atomique dure 10^{-6} secondes ?

Exercice 3 : La famille NP

Question 1 :

Que signifie qu'un problème p est dans P ?

Question 2 :

Que signifie qu'un problème p est dans NP ?

Question 3 :

Montrer que le problème de l'Ensemble Stable appartient à NP.

Question 4 :

Que signifie qu'un problème p est NP-Complet ?

Exercice 4 : Réduction

Les problèmes **Couverture par Sommets (CS)** et **Ensemble Dominant (ED)** sont des cousins du problème de l'ensemble stable. Tous sont NP-complets, mais il faut le démontrer.

Problème CS (Couverture par Sommets)

Donnée : un graphe $G = (V, E)$ et un entier k . **Question :** existe-t-il un ensemble $C \subseteq V$ de taille k tel que chaque arête ait au moins une extrémité dans C ?

Question 1 :

Construire une **réduction polynomiale** de l'Ensemble Stable vers la Couverture par Sommets. Montrer que si l'on savait résoudre CS en temps polynomial, on saurait aussi résoudre l'Ensemble Stable.

Problème ED (Ensemble Dominant)

Un ensemble dominant (ou dominating set en anglais) d'un graphe $G = (S, A)$ est un sous-ensemble D de l'ensemble S des sommets tel que tout sommet qui n'appartient pas à D possède au moins une arête commune avec un sommet de D .

Donnée : un graphe $G = (V, E)$ et un entier k . **Question :** existe-t-il un ensemble $D \subseteq V$ tel que tout sommet de G soit soit dans D , soit adjacent à un sommet de D ?

Question 2 :

Donner une réduction polynomiale de CS vers ED.

Question 3 :

En quoi la décidabilité d'un problème (par une machine de Turing) diffère-t-elle de sa complexité (en nombre d'étapes) ? Donner un exemple concret dans le cas de Graphopolis.

Partie II, tableau dynamique

On considère un tableau dynamique (comme `ArrayList` en Java ou `list` en Python) qui redimensionne automatiquement sa capacité lorsqu'il n'y a plus de place disponible.

Règles de croissance

- Capacité initiale : 1
- Lorsque le tableau est plein et qu'un nouvel élément doit être inséré :
 - Si la capacité actuelle est **paire** : nouvelle capacité = capacité $\times$ 2
 - Si la capacité actuelle est **impaire** : nouvelle capacité = capacité $\times$ 3
- Après redimensionnement, tous les éléments sont recopiés dans le nouveau tableau

Modèle de coût

Pour chaque insertion :

- Si **pas de redimensionnement** : coût = 1 (insertion seule)
- Si **redimensionnement** : coût = ancienne capacité + 1
(ancienne capacité pour recopier tous les éléments + 1 pour insérer le nouvel élément)

Questions

1. Calculer le coût de chaque insertion successivement jusqu'à la 10ème insertion.
2. Donner le coût total pour N insertions.
3. Déterminer la complexité d'une insertion.
4. Déterminer la complexité d'une insertion.